

实验二：编码器、译码器、显示译码器

实验日期：2025.10.31

姓名：何宗霖

学号：23374275

班级：231516

任课教师：李家军

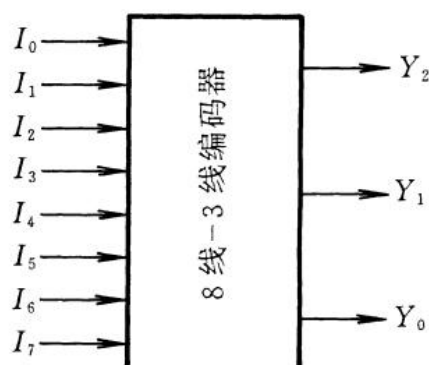
一、实验目的

1. 熟练使用 Verilog 语言、Quartus II 软件，掌握基本的设计思想，软件环境的参数配置，仿真，管脚分配，下载等基本操作，掌握数字电路设计的基本流程；
2. 了解普通编码器、普通译码器、优先编码器、集成电路显示译码器 7448 的实验原理
3. 代码实现 8-3 普通编码器、8-3 优先编码器，实现 8-3 普通译码器并将译码结果驱动七段显示译码器进行显示，并分别对三个实验进行仿真。

二、实验原理

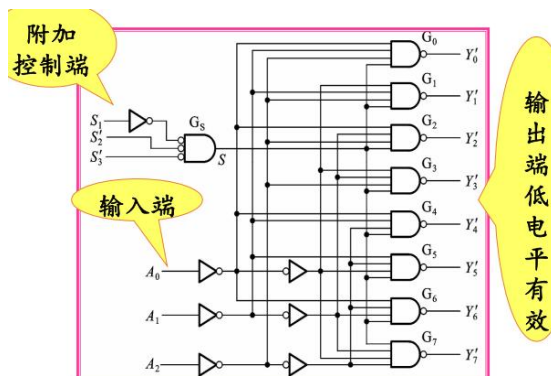
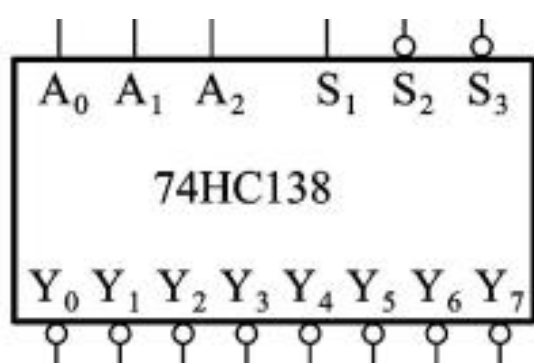
1. 普通 8-3 编码器程序：

对某个输入端出现有效信号的状态进行编码（用二进制码表示）。任何时刻，只有一个输入端有有效信号。

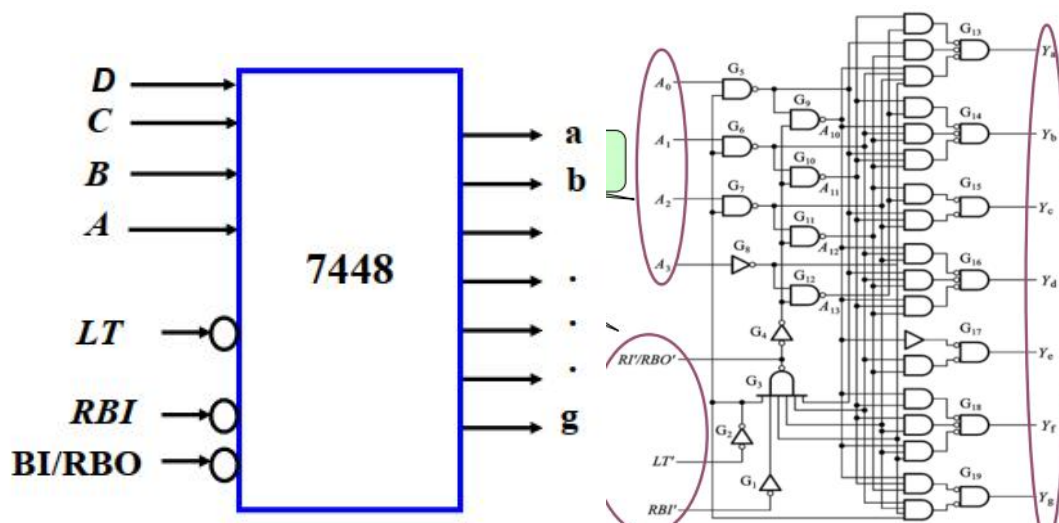


2. 74138 集成译码器：

译码器是一种数字电路，它的功能是将一种形式的信息代码转换成另一种更易于理解或处理的形式。



3. 集成电路显示译码器 7448:



三、实验内容

1. (必做) 使用 Verilog 语言设计一个 8-3 普通编码器程序;
2. (必做) 使用 Verilog 语言设计一个 8-3 普通译码器程序;
3. (选做) 使用 Verilog 语言设计一个 8-3 优先编码器程序;

四、硬件资源

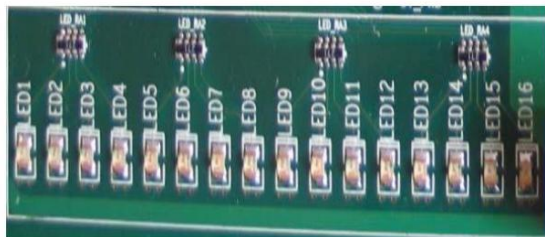
1. 实验用 FPGA 芯片: 在本实验中, 使用了 Altera (Intel) 公司 Cyclone IV 系列 FPGA 芯片, 型号为 EP4CE10F17C8N, 有近 10k 逻辑资源, 速度等级为 8, FBGA 封装, 256 个 IO 引脚。

资源	EP4CE6	EP4CE10	EP4CE15	EP4CE22	EP4CE30	EP4CE40	EP4CE55	EP4CE75	EP4CE115
逻辑单元 (LE)	6, 272	10, 320	15, 408	22, 320	28, 848	39, 600	55, 856	75, 408	114, 480
嵌入式存储器 (Kbits)	270	414	504	594	594	1, 134	2, 340	2, 745	3, 888
嵌入式 18 × 18 乘法器	15	23	56	66	66	116	154	200	266
通用 PLL	2	2	4	4	4	4	4	4	4
全局时钟网络	10	10	20	20	20	20	20	20	20
用户 I/O 块	8	8	8	8	8	8	8	8	8
最大用户 I/O (注释 1)	179	179	343	153	532	532	374	426	528

【注】: EP4CE10F17C8N 为 PPT 上所写, 实际 Quartus 用的是 EP4CE115F23C7 型号的芯片, PPT 中并未作出介绍

2.LED 指示灯:

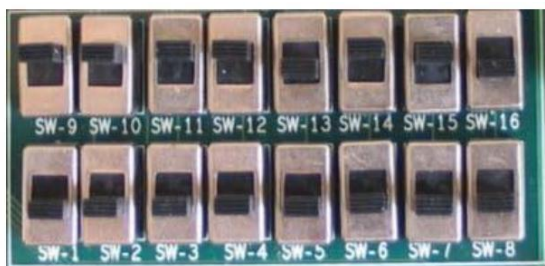
开发平台上有 2x8 共 16 个 LED 指示灯, 便于对用户的操作给出提示, 使用户得到相应的反馈信息, 也可验证设计的正确性。注意为低电平时亮。



LED1	0V_D5	由开关 VLPO 选择	CON1. 2	C10
LED2	0V_D6	由开关 VLPO 选择	CON1. 3	E9
LED3	0V_D7	由开关 VLPO 选择	CON1. 4	D10
LED4	0V_D8	由开关 VLPO 选择	CON1. 5	V13
LED5	0V_D9	由开关 VLPO 选择	CON1. 6	V14
LED6	VSYNC	由开关 VLPO 选择	CON1. 7	Y17
LED7	PCLK	由开关 VLPO 选择	CON1. 8	W17
LED8	0V_RES	由开关 VLPO 选择	CON1. 9	W19

3.拨码开关:

开发平台上有二组 16 只拨码开关输入，开关拨置上为逻辑“1”电平，开关拨置下为逻辑“0”电平。



SW1			CON1. 33	M16
SW2			CON1. 34	R19
SW3			CON1. 54	AA14
SW4			CON1. 61	AA1
SW5			CON2. 62	A3
SW6			CON2. 61	B4
SW7			CON2. 60	C4
SW8			CON2. 59	E6

4.对应关系:

在本次基础实验中，管脚分配如下表

设计端口	EP4CE115F23C7	开发平台模块
I7	M16	SW1
I6	R19	SW2
I5	AA14	SW3
I4	AA1	SW4
I3	A3	SW5
I2	B4	SW6
I1	C4	SW7
I0	E6	SW8
Y2	C10	LED1
Y1	E9	LED2
Y0	D10	LED3

在本次附加实验 1 中，管脚分配如下表

设计端口	EP4CE115F23C7	开发平台模块
A2	M16	SW1
A1	R19	SW2
A0	AA14	SW3
Y7	C10	LED1
Y6	E9	LED2
Y5	D10	LED3
Y4	V13	LED4
Y3	V14	LED5

Y2	Y17	LED6
Y1	W17	LED7
Y0	W19	LED8

4. 外设

➤ 数码管



seg控制一个数码管哪一段是否发光

设计端口	引脚分配	开发平台模块
Seg[7]	U21	8xSEG LA
Seg[6]	R22	8xSEG LB
Seg[5]	T17	8xSEG LC
Seg[4]	R21	8xSEG LD
Seg[3]	P22	8xSEG LE
Seg[2]	R17	8xSEG LF
Seg[1]	R20	8xSEG LG
Seg[0] (高使能)	P20	8xSEG LH
Sel[1]	AB17	8xSEG DS2
Sel[0] (低使能)	T18	8xSEG DS1

数码管段选

Sel为数码管使能端

控制某一个数码管是否被使能

sel[7]	B12	W21	8xSEG DS8
sel[6]	K20	AA20	8xSEG DS7
sel[5]	J18	AA19	8xSEG DS6
sel[4]	H11	AB20	8xSEG DS5
sel[3]	L7	AA17	8xSEG DS4
sel[2]	L8	AB16	8xSEG DS3
sel[1]	K9	AB17	8xSEG DS2
sel[0]	H6	T18	8xSEG DS1

数码管位选

五、实验过程与结果

1. 用 Verilog 语言设计一个普通 8-3 编码器程序。

实验代码：

```

module mux_3b(
    input [7:0] X,
    output reg [2:0] y
);

always @(X) begin
    case(X)
        8'b00000001: y <= 3'b000; //b00000001是0
        8'b00000010: y <= 3'b001;
        8'b00000100: y <= 3'b010;
        8'b00001000: y <= 3'b011;
        8'b00010000: y <= 3'b100;
        8'b00100000: y <= 3'b101;
        8'b01000000: y <= 3'b110;
        8'b10000000: y <= 3'b111;
        default: y <= 3'b000;
    endcase
end

endmodule

```

编写测试代码进行仿真，本地结果如下：

```

PS D:\VLSI-project\experiment3\1> iverilog -o mux_3b.out mux_3b.v mux_3b_tb.v
PS D:\VLSI-project\experiment3\1> vvp mux_3b.out
VCD info: dumpfile mux_3b.vcd opened for output.
X = 00000000, y = 000
X = 00000001, y = 000
X = 00000010, y = 001
X = 00000100, y = 010
X = 00001000, y = 011
X = 00010000, y = 100
X = 00100000, y = 101
X = 01000000, y = 110
X = 10000000, y = 111

```

仿真结果正确后，修改程序使输出低电平有效，将程序烧录到实验平台上。

```
1 module mux_3b(  
2     input [7:0] X,  
3     output reg [2:0] y  
4 );  
5  
6 always @(X) begin  
7     case(X)  
8         8'b00000001: y <= ~3'b000;  
9         8'b00000010: y <= ~3'b001;  
10        8'b00000100: y <= ~3'b010;  
11        8'b00001000: y <= ~3'b011;  
12        8'b00010000: y <= ~3'b100;  
13        8'b00100000: y <= ~3'b101;  
14        8'b01000000: y <= ~3'b110;  
15        8'b10000000: y <= ~3'b111;  
16        default: y <= ~3'b000;  
17    endcase  
18 end  
19  
20 endmodule
```

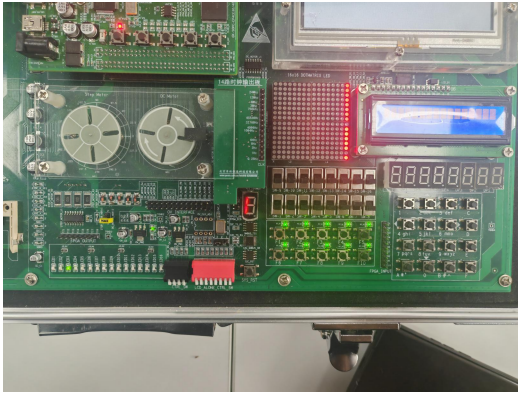
本地验证：

```
● PS D:\VLSI-project\experiment3> cdiverilog -o mux_3b.out mux_3b.v mux_3b_tb.v -project\experiment3\1>  
● PS D:\VLSI-project\experiment3\1> vvp mux_3b.out  
VCD info: dumpfile mux_3b.vcd opened for output.  
X = 00000000, y = 111  
X = 00000001, y = 111  
X = 00000010, y = 110  
X = 00000100, y = 101  
X = 00001000, y = 100  
X = 00010000, y = 011  
X = 00100000, y = 010  
X = 01000000, y = 001  
X = 10000000, y = 000
```

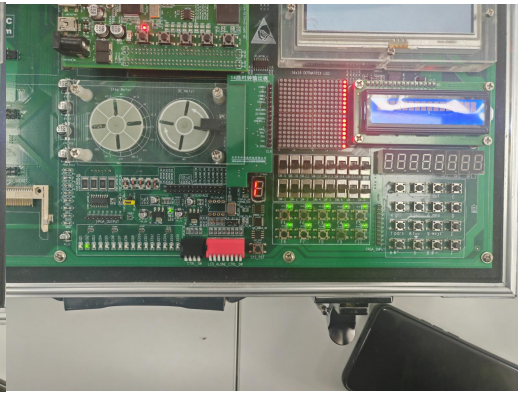
管脚绑定情况界面：

Named: * Edit: X										
Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
X[7]	Input	PIN_M16	5	B5_N2	PIN_M16	2.5 V		8mA (...ault)		
X[6]	Input	PIN_R19	5	B5_N1	PIN_R19	2.5 V ...fault)		8mA (...ault)		
X[5]	Input	PIN_AA14	4	B4_N2	PIN_AA14	2.5 V ...fault)		8mA (...ault)		
X[4]	Input	PIN_AA1	2	B2_N1	PIN_AA1	2.5 V ...fault)		8mA (...ault)		
X[3]	Input	PIN_A3	8	B8_N2	PIN_A3	2.5 V ...fault)		8mA (...ault)		
X[2]	Input	PIN_B4	8	B8_N1	PIN_B4	2.5 V ...fault)		8mA (...ault)		
X[1]	Input	PIN_C4	8	B8_N2	PIN_C4	2.5 V ...fault)		8mA (...ault)		
X[0]	Input	PIN_E6	8	B8_N2	PIN_E6	2.5 V ...fault)		8mA (...ault)		
y[2]	Output	PIN_C10	8	B8_N0	PIN_C10	2.5 V ...fault)		8mA (...ault)	2 (default)	
y[1]	Output	PIN_E9	8	B8_N0	PIN_E9	2.5 V ...fault)		8mA (...ault)	2 (default)	
y[0]	Output	PIN_D10	8	B8_N0	PIN_D10	2.5 V ...fault)		8mA (...ault)	2 (default)	
<<new node>>										

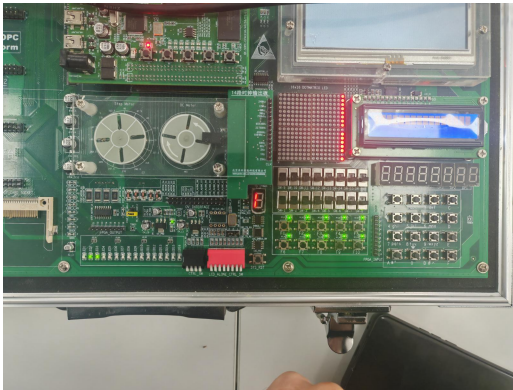
最终结果：



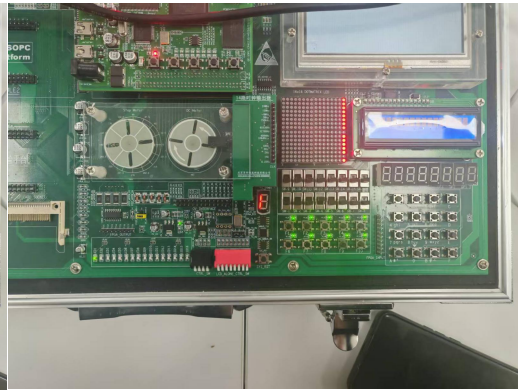
输入 00000010, 输出 001



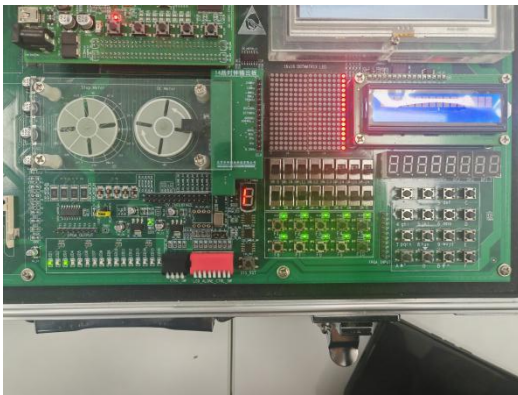
输入 00000100, 输出 010



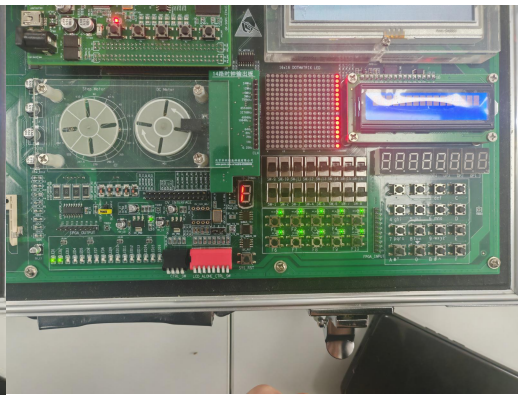
输入 00001000, 输出 011



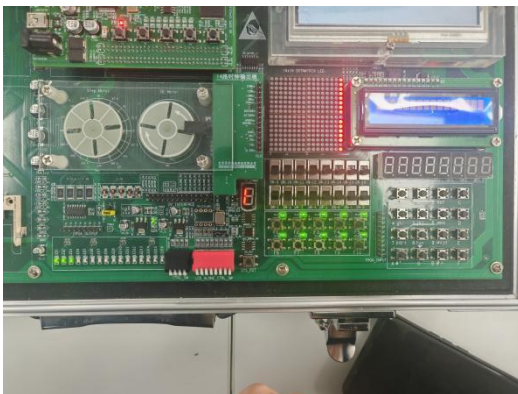
输入 00010000, 输出 100



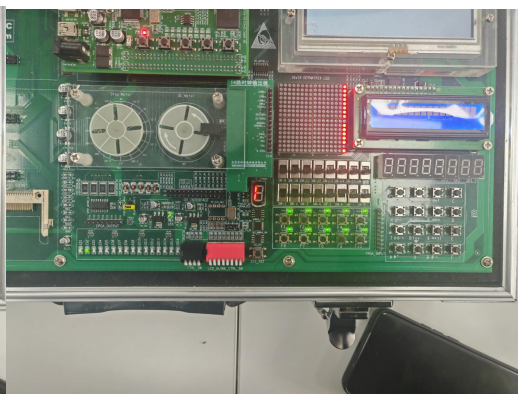
输入 00100000, 输出 101



输入 01000000, 输出 110



输入 10000000, 输出 111



输入 00000000, 输出 000

2.用 Verilog 语言设计一个普通 3-8 译码器程序

实验代码:

```
1 module demux_3b(  
2     input [2:0] x,  
3     output [7:0] y,  
4     output reg[7:0] seg,  
5     output sel  
6 );  
7  
8     assign y = 8'b1 << x;  
9  
10    always@(*) begin  
11        case(x)  
12            3'd0: seg <= 8'b11111100;  
13            3'd1: seg <= 8'b01100000;  
14            3'd2: seg <= 8'b11011010;  
15            3'd3: seg <= 8'b11110010;  
16            3'd4: seg <= 8'b01100110;  
17            3'd5: seg <= 8'b10110110;  
18            3'd6: seg <= 8'b10111110;  
19            3'd7: seg <= 8'b11100000;  
20            default: seg <= 8'b00000000;  
21        endcase  
22    end  
23  
24    assign sel = 1'b0;  
25    //如果sel为0, 则表示demux处于工作状态  
26    //如果sel为1, 则表示demux处于禁止状态, 所有输出均为0  
27  
28 endmodule
```

编写测试代码进行仿真, 本地结果如下:

```
PS D:\VLSI-project\experiment3\2> vvp demux_3b.out  
VCD info: dumpfile demux_3b.vcd opened for output.  
x = 000, y = 00000001, seg = 11111100, sel = 0  
x = 001, y = 00000010, seg = 01100000, sel = 0  
x = 010, y = 00000100, seg = 11011010, sel = 0  
x = 011, y = 00001000, seg = 11110010, sel = 0  
x = 100, y = 00010000, seg = 01100110, sel = 0  
x = 101, y = 00100000, seg = 10110110, sel = 0  
x = 110, y = 01000000, seg = 10111110, sel = 0  
x = 111, y = 10000000, seg = 11100000, sel = 0  
demux_3b_tb.v:28: $finish called at 80 (1ns)
```

仿真结果正确后, 修改程序使输出低电平有效, 将程序烧录到实验平台上。

```
demux_3b_tb.v demux_3bv X  
1 module demux_3b(  
2     input [2:0] x,  
3     output [7:0] y,  
4     output reg[7:0] seg,  
5     output sel  
6 );  
7     wire [7:0] res= 8'b1 << x;  
8     assign y = ~res;  
9  
10    always@(*) begin  
11        case(x)  
12            3'd0: seg <= 8'b11111100;  
13            3'd1: seg <= 8'b01100000;  
14            3'd2: seg <= 8'b11011010;  
15            3'd3: seg <= 8'b11110010;  
16            3'd4: seg <= 8'b01100110;  
17            3'd5: seg <= 8'b10110110;  
18            3'd6: seg <= 8'b10111110;  
19            3'd7: seg <= 8'b11100000;  
20            default: seg <= 8'b00000000;  
21        endcase  
22    end  
23  
24    assign sel = 1'b0;  
25    //如果sel为0, 则表示demux处于工作状态  
26    //如果sel为1, 则表示demux处于禁止状态, 所有输出均为0  
27  
28 endmodule  
29
```


本地验证:

```

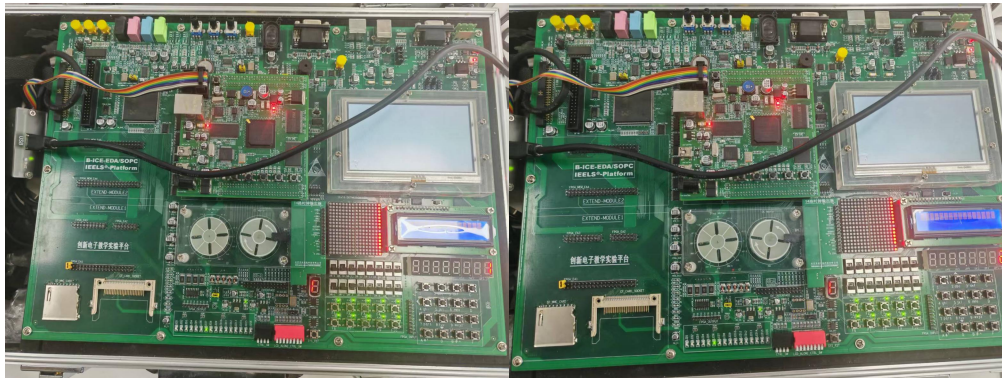
PS D:\VLSI-project\experiment3\2> iverilog -o demux_3b.out demux_3b.v demux_3b_tb.v
PS D:\VLSI-project\experiment3\2> vvp demux_3b.out
VCD info: dumpfile demux_3b.vcd opened for output.
x = 000, y = 00000001, seg = 00000011, sel = 0
x = 001, y = 00000010, seg = 10011111, sel = 0
x = 010, y = 00000100, seg = 00100101, sel = 0
x = 011, y = 00001000, seg = 00001101, sel = 0
x = 100, y = 00010000, seg = 10011001, sel = 0
x = 101, y = 00100000, seg = 01001001, sel = 0
x = 110, y = 01000000, seg = 01000001, sel = 0
x = 111, y = 10000000, seg = 00011111, sel = 0

```

管脚绑定情况界面:

Node Name	Direction	Location	Y/C Bank	Pin Group	Net Label	Y/C Bank	Net Ref	Pin Group	Y/C Bank	Net Ref
seq[7]	Output	PIN_U21	5	B5_N1	PIN_U21	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[6]	Output	PIN_R22	5	B5_N1	PIN_R22	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[5]	Output	PIN_T17	5	B5_N2	PIN_T17	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[4]	Output	PIN_R21	5	B5_N1	PIN_R21	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[3]	Output	PIN_P22	5	B5_N1	PIN_P22	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[2]	Output	PIN_R17	5	B5_N2	PIN_R17	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[1]	Output	PIN_R20	5	B5_N1	PIN_R20	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[0]	Output	PIN_P20	5	B5_N1	PIN_P20	2.5 V ...fault		8mA (...ault)	2 (default)	
sel	Output	PIN_AB17	4	B4_N1	PIN_AB17	2.5 V ...fault		8mA (...ault)	2 (default)	
x[2]	Input	PIN_M16	5	B5_N2	PIN_M16	2.5 V ...fault		8mA (...ault)		
x[1]	Input	PIN_R19	5	B5_N1	PIN_R19	2.5 V ...fault		8mA (...ault)		
x[0]	Input	PIN_AA14	4	B4_N2	PIN_AA14	2.5 V ...fault		8mA (...ault)		
y[7]	Output	PIN_C10	8	B8_N0	PIN_C10	2.5 V ...fault		8mA (...ault)	2 (default)	
y[6]	Output	PIN_E9	8	B8_N0	PIN_E9	2.5 V ...fault		8mA (...ault)	2 (default)	
y[5]	Output	PIN_D10	8	B8_N0	PIN_D10	2.5 V ...fault		8mA (...ault)	2 (default)	
y[4]	Output	PIN_V13	4	B4_N2	PIN_V13	2.5 V ...fault		8mA (...ault)	2 (default)	
y[3]	Output	PIN_V14	4	B4_N1	PIN_V14	2.5 V ...fault		8mA (...ault)	2 (default)	
y[2]	Output	PIN_Y17	4	B4_N1	PIN_Y17	2.5 V ...fault		8mA (...ault)	2 (default)	
y[1]	Output	PIN_W17	4	B4_N1	PIN_W17	2.5 V ...fault		8mA (...ault)	2 (default)	
y[0]	Output	PIN_W19	5	B5_N2	PIN_W19	2.5 V ...fault		8mA (...ault)	2 (default)	
<<new node>>										

最终结果:



输入 001, 输出 00000010

输入 010, 输出 00000100



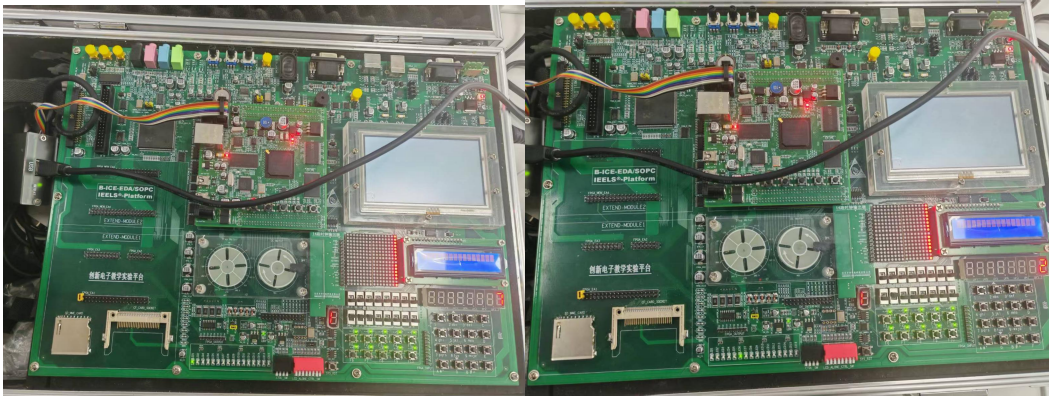
输入 011, 输出 00001000

输入 100, 输出 00010000



输入 101, 输出 00100000

输入 110, 输出 01000000



输入 111, 输出 10000000

输入 000, 输出 00000001

3. 用 Verilog 语言设计一个 8-3 优先编码器程序 实验代码:

```

1  module prior_mux_3b(
2      input [7:0] X,
3      output reg [2:0] y
4  );
5
6  always @(X) begin
7      casex(X)
8          8'b00000001: y <= 3'b000;
9          8'b0000001x: y <= 3'b001;
10         8'b000001xx: y <= 3'b010;
11         8'b00001xxx: y <= 3'b011;
12         8'b0001xxxx: y <= 3'b100;
13         8'b001xxxxx: y <= 3'b101;
14         8'b01xxxxxx: y <= 3'b110;
15         8'b1xxxxxxx: y <= 3'b111;
16         default: y <= 3'b000;
17     endcase
18 end
19
20 endmodule

```

编写测试代码进行仿真，本地结果如下:

```

● PS D:\VLSI-project\experiment3\addition1> iverilog -o prior_mux_3b.out prior_mux_3b.v prior_mux_3b_tb.v
● PS D:\VLSI-project\experiment3\addition1> vvp prior_mux_3b.out
VCD info: dumpfile prior_mux_3b.vcd opened for output.
X = 00000000, y = 000
X = 00000001, y = 000
X = 00000010, y = 001
X = 00000011, y = 001
X = 00000100, y = 010
X = 00000111, y = 010
X = 00001000, y = 011
X = 00001111, y = 011
X = 00010000, y = 100
X = 00010001, y = 100
X = 00010100, y = 100
X = 00010111, y = 100
X = 00011000, y = 100
X = 00011001, y = 100
X = 00011010, y = 100
X = 00011101, y = 100
X = 00100000, y = 101
X = 01000000, y = 110
X = 10000001, y = 111
prior_mux_3b_tb.v:37: $finish called at 190 (1ns)

```

仿真结果正确后，修改程序使输出低电平有效，将程序烧录到实验平台上。

```

1  module prior_mux_3b(
2      input [7:0] X,
3      output reg [2:0] y
4  );
5
6  always @(X) begin
7      casex(X)
8          8'b00000001: y <= ~3'b000;
9          8'b0000001x: y <= ~3'b001;
10         8'b000001xx: y <= ~3'b010;
11         8'b00001xxx: y <= ~3'b011;
12         8'b0001xxxx: y <= ~3'b100;
13         8'b001xxxxx: y <= ~3'b101;
14         8'b01xxxxxx: y <= ~3'b110;
15         8'b1xxxxxxx: y <= ~3'b111;
16         default: y <= ~3'b000;
17     endcase
18 end
19
20 endmodule
21

```

本地验证：

```

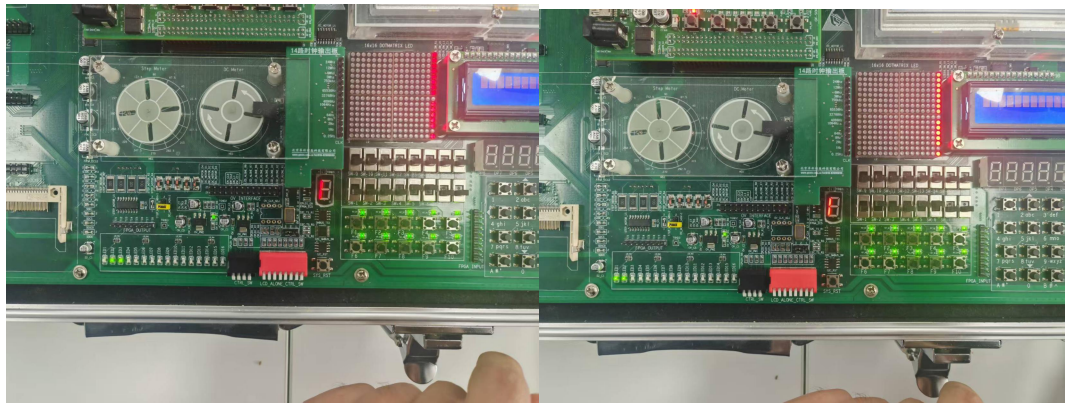
● PS D:\VLSI-project\experiment3\addition1> iverilog -o prior_mux_3b.out prior_mux_3b.v prior_mux_3b_tb.v
● PS D:\VLSI-project\experiment3\addition1> vvp prior_mux_3b.out
VCD info: dumpfile prior_mux_3b.vcd opened for output.
X = 00000000, y = 111
X = 00000001, y = 111
X = 00000010, y = 110
X = 00000011, y = 110
X = 00000100, y = 101
X = 00000111, y = 101
X = 00001000, y = 100
X = 00001111, y = 100
X = 00010000, y = 011
X = 00010001, y = 011
X = 00010100, y = 011
X = 00010111, y = 011
X = 00011000, y = 011
X = 00011001, y = 011
X = 00011010, y = 011
X = 00011101, y = 011
X = 00100000, y = 010
X = 01000000, y = 001
X = 10000001, y = 000

```


管脚绑定情况界面：

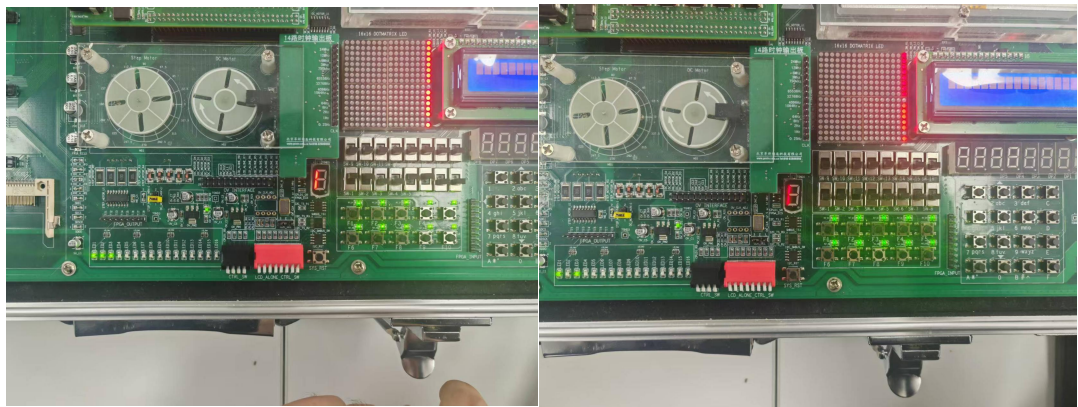
Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
X[7]	Input	PIN_M16	5	B5_N2	PIN_M16	2.5 V ...fault		8mA (...ault)		
X[6]	Input	PIN_R19	5	B5_N1	PIN_R19	2.5 V ...fault		8mA (...ault)		
X[5]	Input	PIN_AA14	4	B4_N2	PIN_AA14	2.5 V ...fault		8mA (...ault)		
X[4]	Input	PIN_AA1	2	B2_N1	PIN_AA1	2.5 V ...fault		8mA (...ault)		
X[3]	Input	PIN_A3	8	B8_N2	PIN_A3	2.5 V ...fault		8mA (...ault)		
X[2]	Input	PIN_B4	8	B8_N1	PIN_B4	2.5 V ...fault		8mA (...ault)		
X[1]	Input	PIN_C4	8	B8_N2	PIN_C4	2.5 V ...fault		8mA (...ault)		
X[0]	Input	PIN_E6	8	B8_N2	PIN_E6	2.5 V ...fault		8mA (...ault)		
y[2]	Output	PIN_C10	8	B8_N0	PIN_C10	2.5 V ...fault		8mA (...ault)	2 (default)	
y[1]	Output	PIN_E9	8	B8_N0	PIN_E9	2.5 V ...fault		8mA (...ault)	2 (default)	
y[0]	Output	PIN_D10	8	B8_N0	PIN_D10	2.5 V ...fault		8mA (...ault)	2 (default)	

最终结果：



输入 00001100，输出 011

输入 01001100，输出 110



输入 11001100，输出 111

输入 00100000，输出 101

六、实验仿真波形

1.编写 mux_3b_tb.v 代码

```
1  `timescale 1ns/1ns//照常写
2
3  //mux_3b_tb后面需要加括号
4  module mux_3b_tb();
5
6      reg [7:0] X;
7      wire [2:0] y;
8
9      initial begin
10
11          $dumpfile("mux_3b.vcd");//修改名字
12          $dumpvars(0, mux_3b_tb);//照常写
13          $monitor("X = %b, y = %b", X, y);
14          //监视信号变化，打印输出
15          // 初始化输入
16          X=8'b00000000; #10;
17
18          X=8'b00000001; #10;
19          X=8'b00000010; #10;
20          X=8'b00000100; #10;
21          X=8'b00001000; #10;
22          X=8'b00010000; #10;
23          X=8'b00100000; #10;
24          X=8'b01000000; #10;
25          X=8'b10000000; #10;
26
27          $finish;
28          // $stop;暂停仿真，等待输入
29      end
30
31      mux_3b mux_3b_obj (.X(X), .y(y));
32
33  endmodule
34
```

1) 编译文件

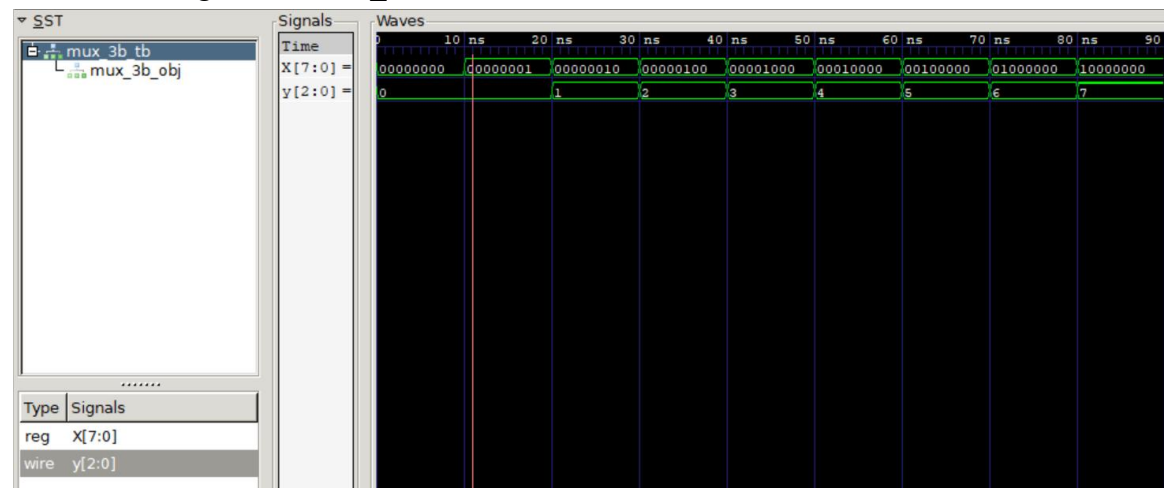
命令行: iverilog -o mux_3b.out mux_3b.v mux_3b_tb.v

2) 生成 .vcd 波形文件

命令行: vvp mux_3b.out

3) 观察波形

命令行: gtkwave mux_3b.vcd



2.编写 demux_3b_tb.v 代码

```
1  `timescale 1ns/1ns//照常写
2
3  //demux_3b_tb后面需要加括号
4  module demux_3b_tb();
5
6      reg [2:0] x;
7      wire [7:0] y;
8      wire [7:0] seg;
9      wire sel;
10
11     initial begin
12
13         $dumpfile("demux_3b.vcd");//修改名字
14         $dumpvars(0, demux_3b_tb);//照常写
15         $monitor("x = %b, y = %b, seg = %b, sel = %b", x, y, seg, sel);
16         //监视信号变化, 打印输出
17         // 初始化输入
18         x=3'b000; #10;
19
20         x=3'b001; #10;
21         x=3'b010; #10;
22         x=3'b011; #10;
23         x=3'b100; #10;
24         x=3'b101; #10;
25         x=3'b110; #10;
26         x=3'b111; #10;
27
28         $finish;
29         // $stop;暂停仿真, 等待输入
30     end
31
32     demux_3b demux_3b_obj (.x(x), .y(y), .seg(seg), .sel(sel));
33
34 endmodule
35
```

1) 编译文件

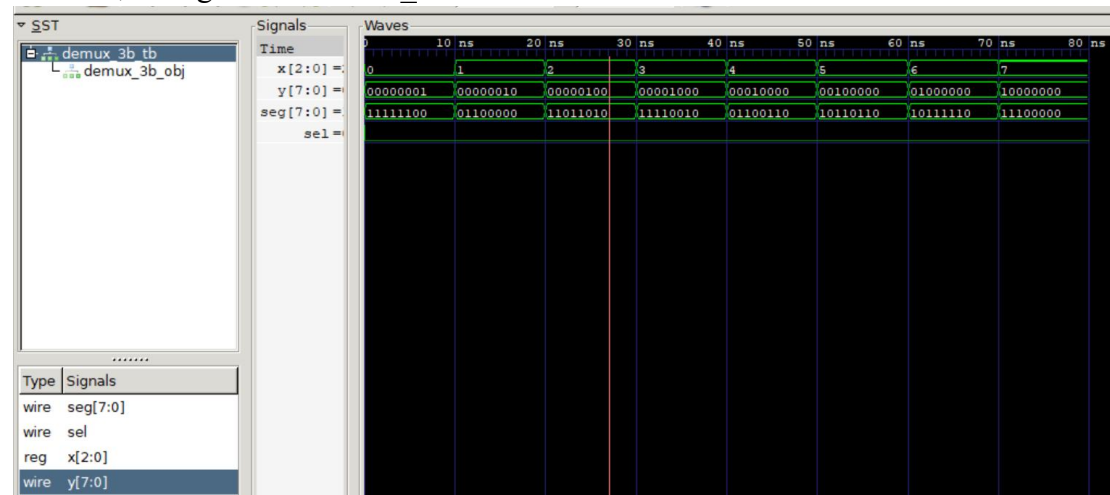
命令行: iverilog -o demux_3b.out demux_3b.v demux_3b_tb.v

2) 生成 .vcd 波形文件

命令行: vvp demux_3b.out

3) 观察波形

命令行: gtkwave demux_3b.vcd



七、附加实验仿真波形

1.附加实验 1:

编写 prior_mux_3b_tb.v 代码

```
prior_mux_3b_tb.v X
1 | timescale 1ns/1ns//照常写
2
3 //prior_mux_3b_tb后面需要加括号
4 module prior_mux_3b_tb();
5
6     reg [7:0] X;
7     wire [2:0] y;
8
9     initial begin
10
11         $dumpfile("prior_mux_3b.vcd");//修改名字
12         $dumpvars(0, prior_mux_3b_tb);//照常写
13         $monitor("X = %b, y = %b", X, y);
14         //监视信号变化, 打印输出
15         // 初始化输入
16         X=8'b00000000; #10;
17
18         X = 8'b00000001; #10;
19         X = 8'b00000010; #10;
20         X = 8'b00000011; #10;
21         X = 8'b00000100; #10;
22         X = 8'b00000111; #10;
23         X = 8'b00001000; #10;
24         X = 8'b00001111; #10;
25         X = 8'b00010000; #10;
26         X = 8'b00010001; #10;
27         X = 8'b00010100; #10;
28         X = 8'b00010111; #10;
29         X = 8'b00011000; #10;
30         X = 8'b00011001; #10;
31         X = 8'b00011010; #10;
32         X = 8'b00011101; #10;
33         X = 8'b00100000; #10;
34         X = 8'b01000000; #10;
35         X = 8'b10000001; #10;
36
37         $finish;
38         // $stop;暂停仿真, 等待输入
39     end
40
41     prior_mux_3b prior_mux_3b_obj (.X(X), .y(y));
42
43 endmodule
```

1) 编译文件

命令行: iverilog -o prior_mux_3b.out prior_mux_3b.v prior_mux_3b_tb.v

2) 生成 .vcd 波形文件

命令行: vvp prior_mux_3b.out

3) 观察波形

命令行: gtkwave prior_mux_3b.vcd

